Meteorite Atmospheric Entry Trajectory Simulation

Rohan Tummala (33426104) | GitHub: https://github.com/Rohan-Tummala/meteorite-trajectory

1. Engineering Problem

When a meteoroid enters the atmosphere, its trajectory, deceleration and mass loss are governed by drag and ablation as it falls through the air with rapidly changing density. For a small fraction of falls, fragments survive all the way to the ground, and the point where they land can theoretically be predicted from the observed entry trajectory. This matters in practice since meteorite recovery teams use predicted strewn fields to decide where to search after a fireball, and the same physics applies to spacecraft re-entry and asteroid airburst studies.

This project simulates the atmospheric entry of a meteoroid as a 3D point-mass dynamical system, integrating the coupled equations of motion for altitude, speed, flight-path angle, position and mass from a given entry state down to ground impact. The simulation is applied to two real, independently documented falls, Bunburra Rockhole (Australia, Spurny et al. 2012) and Cavezzo (Italy, Gardiol et al. 2021), using each paper's reported entry conditions as the starting point. The predicted impact point is then compared against the fall's actual recovered location, which is a genuine engineering validation question. Does a simplified, computationally cheap entry model produce a strewn-field prediction that is actually useful, or does it miss by enough that it would not have helped a recovery search?

The aim is to test whether this simple model can predict real meteorite recovery locations and to separate errors caused by numerical integration from those caused by the physical assumptions. Fixed step and adaptive solvers are compared, and Monte Carlo sampling is used to estimate uncertainty in the impact location.

The scope is deliberately narrow. The model tracks 3D position (altitude, latitude, longitude) but freezes heading, so it does not resolve cross-range aerodynamic drift beyond a simple wind correction, and treats the meteoroid as a single non-rotating, non-fragmenting body with constant density. It does not model shock-layer physics, luminous effect, lift, or detailed ablation physics beyond one empirical ablation coefficient.

2. Inputs, Outputs and Data

Each simulation run takes a seven-element entry state vector, altitude (m), speed (m/s), flight-path angle (rad, measured below horizontal, negative descending), latitude (rad), longitude (rad), heading (rad) and mass (kg), plus a dictionary of physical model parameters (drag coefficient, ablation coefficient, meteoroid bulk density, and related constants.) For the two case studies, the entry state is not freely chosen. It is taken directly from the published entry conditions in Spurny et al. (2012) for Bunburra Rockhole and Gardiol et al. (2021) for Cavezzo. Model parameters default to values from Ceplecha et al. (1998, Table VII), since neither paper reports a fitted drag/ablation coefficient pair for its own fall.

Table 1 lists the entry conditions and reported uncertainties used for each fall.

Table 1: Nominal entry-state values and reported uncertainties for both meteorite falls, taken from the original observational papers.

Input	Cavezzo	Bunburra Rockhole
Altitude h_0 (m)	75900 ± 200	62830 ± 30
Speed V_0 (m/s)	12200 ± 200	13310 ± 20
Flight-path angle γ (deg)	-68.4 ± 0.3	-31.19 ± 0.03
Latitude ϕ_0 (deg)	44.7342 ± 0.0019	-31.4496 ± 0.0003
Longitude λ (deg)	10.7192 ± 0.0019	129.8272 ± 0.00018
Heading ψ_0 (deg)	58.1 ± 0.2	279.15 ± 0.0348
Mass M_0 (kg)	3.5 ± 0.8	22.1 ± 0.3
Drag coefficient (c_d)	0.92	0.92
Ablation coefficient σ_{abl} $\left(\frac{kg}{J}\right)$	1.3×10^{-8}	1.3×10^{-8}
Density ρ_m $\left(\frac{kg}{m^3}\right)$	3322	2700

Altitude is defined over the tabulated atmosphere model's range, and values outside it are clipped rather than rejected, since a diverging integration (already flagged separately by the solver's own stability checks) can briefly pass through non-physical altitudes before the run is abandoned. Mass must stay positive, since the radius calculation depends on it, and unsupported model names are rejected outright, raising a ValueError, which is checked in the test suite.

Each run produces the trajectory history and impact time, position, speed and remaining mass. Impact coordinates are compared with the recovered meteorite locations. Solver comparisons report impact error and runtime relative to a high-accuracy reference, while 5000 Monte Carlo samples per fall are used to estimate impact probability contours.

3. Computational Approach

The trajectory is governed by a system of seven coupled first-order ODEs (the state derivatives for altitude, speed, flight-path angle, latitude, longitude, heading and mass), following the standard drag/ablation equations of motion from Ceplecha et al. (1998). This is an initial-value problem without a closed-form solution once drag and ablation are both active, so it has to be solved numerically. Numerical integration was chosen over any closed form or linearized approximation because the problem is nonlinear and stiff in places.

With state variables altitude h, speed V, flight-path angle γ, latitude ϕ, longitude λ, heading ψ and mass M, the system is:

$$\dot{h} = Vsin(\gamma) \tag{1}$$

$$\dot{V} = -\frac{\rho c_d A V^2}{2M} - gsin(\gamma) \tag{2}$$

$$\dot{\gamma} = \cos(\gamma)\left(\frac{V}{R_E+h} - \frac{g}{V}\right) \tag{3}$$

$$\dot{\phi} = \frac{Vcos(\gamma)\cos(\psi)}{R_E + h} \tag{4}$$

$$\dot{\lambda} = \frac{Vcos(\gamma)\sin(\psi)}{(R_E + h)\cos(\phi)} \tag{5}$$

$$\dot{\psi} = 0 \tag{6}$$

$$\dot{M} = -\frac{\sigma_{abl} A \rho V^3}{2} \tag{7}$$

$$g(h) = g_0 \left(\frac{R_E}{R_E + h}\right)^2 \tag{8}$$

Equation 8 gives the gravitational acceleration used in Eq (2) and (3). It is an inverse-square, altitude dependent model, with $g_0 = 9.80665 \frac{m}{s^2}$ and $R_E = 6371\ km$.

Where $\rho = \rho(h)$ is atmospheric density, $g = g(h)$ is gravitational acceleration, R_E is Earth's radius, c_d is the drag coefficient, σ_{abl} is the ablation coefficient, and the cross-sectional area $A = \pi r^2$. Meteoroid radius was calculated using Eq (9) which assume a sphere of constant bulk density.

$$r = \left(\frac{3M}{4\pi\rho_m}\right)^{\frac{1}{3}} \tag{9}$$

The first two equations are the standard drag-deceleration and gravity-turn terms, the third and fourth track position over a spherical Earth, the heading equation is zero since heading is not modelled as changing, and the mass-loss equation is the single-body ablation law, active only while the meteoroid is luminous.

Several integration methods are compared rather than assuming one is correct. Since the true analytical solution is unknown for the full coupled system, a tight-tolerance adaptive solver (DOP853, rtol = 10^{-10}, atol = 10^{-12}) is used as a reference, or "truth" solution, an 8th-order adaptive method that reaches very tight tolerances at low cost. Tightening its tolerance from rtol = 10^{-10}, the value used for the reference solution elsewhere in this project, to rtol = 10^{-12} moved the predicted impact point by about 2×10^{-5} m, far below the smallest error reported for any method compared, so its own numerical error is negligible here, though it is still not an exact solution in the mathematical sense. Against that reference, fixed-step RK2 (Heun) and RK4 (classical) are implemented by hand and compared at a range of step sizes, alongside scipy's adaptive RK45 at normal tolerance, holding model parameters constant across every run to isolate the comparison to numerical integration error only. A separate stability analysis, used to justify the piecewise step-size scheme in Section 5, also compares implicit fixed-step methods (Implicit Euler and implicit Heun) against the same explicit methods.

The physical model parameters are fixed per Section 2, not fitted per fall. A fixed maximum step size (5 s) is imposed on the adaptive solver, so its runs stay comparable to the fixed-step methods. Implementation is in Python using scipy.integrate.solve_ivp for the adaptive methods, a small hand-written fixed-step integrator module for RK2/RK4, and an event function that stops integration exactly at ground impact.

The main limitation of this approach is that it validates numerical correctness, not physical accuracy, a method can reproduce the DOP853 reference to high precision while the underlying physical model still diverges from reality. Comparing against both a numerical reference and real recovery data, discussed below, is what separates these two error sources.

4. Validation Method and Results

The hand-written RK2 and RK4 integrators are checked against the exact solution of a simple decay ODE over a range of step sizes, and the fitted global convergence order matches theory closely (RK2 fits order 2.03, RK4 fits order 4.03). The Monte Carlo HPD-threshold logic is checked against hand-computed examples and against a known 2D Gaussian, confirming each contour encloses its stated fraction of samples. The gravity model was also validated against known analytical values. The tabulated atmosphere model is checked for consistency against the COESA (1976) equations it is sampled from, which confirms the interpolation rather than the atmosphere model itself.

The headline validation compares each fall's predicted impact point, and the Monte Carlo probability map built around it, against the real recovered location reported in the source paper, using 5000 random samples drawn from the entry-state uncertainties reported by each fall's own authors (Table 1). This is an appropriate validation method since it is the only ground truth the project has that was not used to tune the model, the inputs come from the paper's reported trajectory conditions, and the recovery coordinates are an independent, physically measured output. The acceptance criterion used is whether the real recovery site falls within the model's predicted 3-sigma (98.9%) highest-posterior-density (HPD) region.

A second, independent check is used for Cavezzo. The model's height vs time curve during luminous flight (0-6s) is compared against reference points digitized directly from Gardiol et al. (2021)'s own Fig. 4a. The model tracks the digitized curve closely over the full luminous-flight window (RMSE 1.50 km, max error 2.30 km, across a height range of 22 to 76 km), which validates the trajectory shape itself, not just the single endpoint used by the impact-location metric. No equivalent published figure exists for Bunburra Rockhole, but both papers report a single terminal point partway through luminous flight, giving a second comparison (Table 2).

Table 2: Model vs. published terminal-point state at end of observed luminous flight, both falls.

Metric	Cavezzo Model	Cavezzo Paper	Bunburra Model	Bunburra Paper
Height (km)	20.94	21.50	26.94	29.59
Velocity (m/s)	4222	4000	8556	5680
Mass (kg)	1.38	1.50	10.55	1.10

Cavezzo tracks closely here, within 3–8% on all three quantities, consistent with its 3.31 km impact miss. Bunburra is off by more, mass most of all, nearly 10 times too high, since its paper reports losing about 60 percent of its mass in a single fragmentation event at 54.9 km altitude, a mechanism this single-body model cannot represent, consistent with its much larger 21.07 km impact miss.

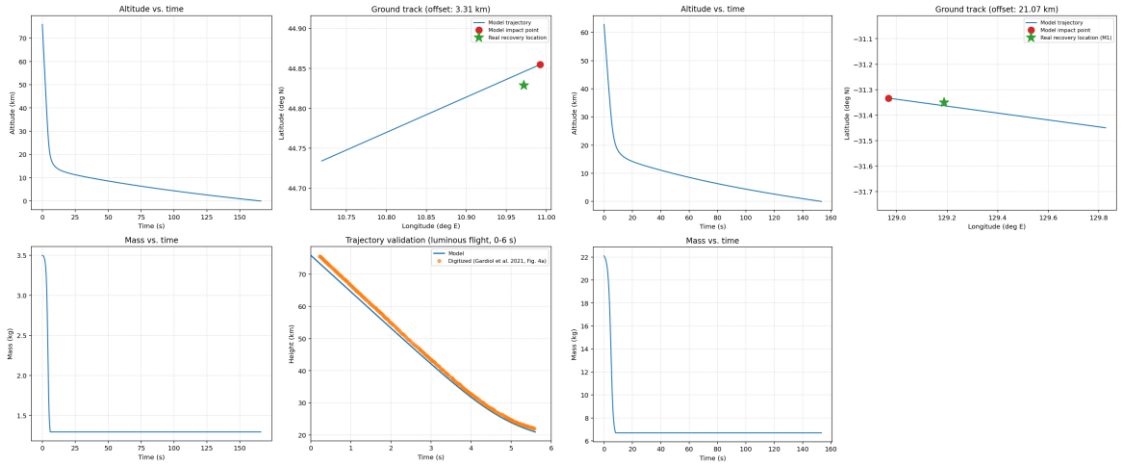


Figure 1(Cavezzo left) & 2(Bunburra right): Simulated trajectory and ground track for each fall vs. reported entry conditions, altitude/mass profiles.

The simulated altitude and mass histories for Cavezzo and Bunburra Rockhole are shown in Figures 1 and 2.

Under the shared default coefficients, the real recovery site falls outside the predicted 3-sigma region for both falls. Cavezzo misses by 3.31 km, Bunburra by 21.07 km. These runs used RK45, and the reason it was chosen is covered in Section 5. The gap is physical-model error, not numerical error. Bunburra's larger discrepancy aligns with the fact that in reality, the body went through relatively high fragmentation (at

54900 m, 60% mass loss) that this non-fragmenting point-mass model cannot represent. On the other hand, Cavezzo has no equivalent event. Both results are tracked as xfail tests rather than silent failures.

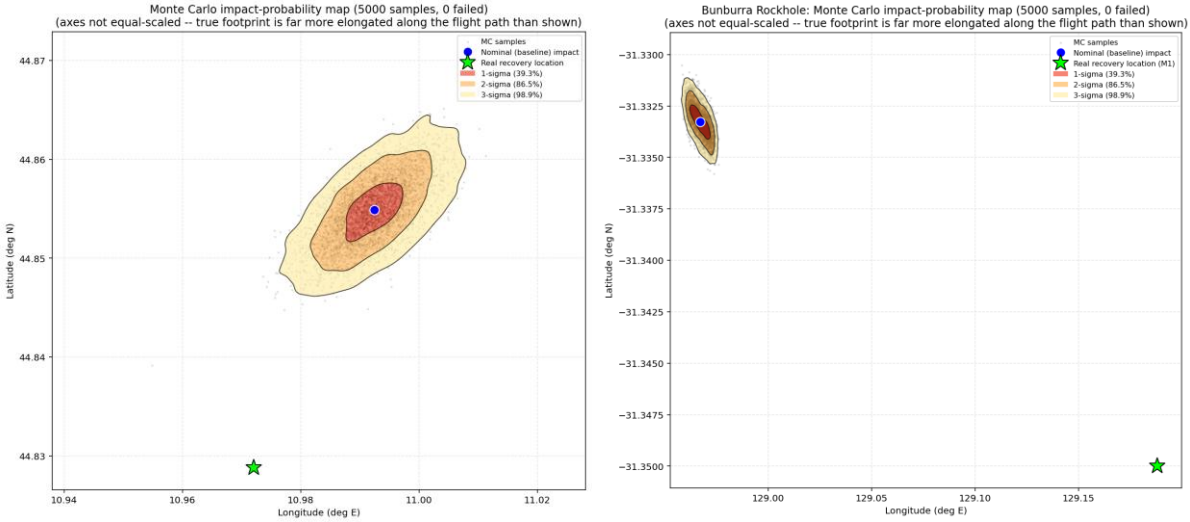


Figure 3(Cavezzo left) & 4(Bunburra right): 1/2/3-sigma HPD probability contours for each fall's Monte Carlo impact-location uncertainty, with nominal prediction and real recovery site both marked.

The Monte Carlo contours in Figures 3 and 4 show that neither recovered location lies within the predicted 3-sigma region.

The comparison is limited to two meteorite falls using the same default coefficients, so it cannot establish how well the model generalizes.

The numerical machinery is checked with a pytest suite of 91 tests across 11 test files, run with “pytest -v” and saved as the project’s test report. 89 pass and 2 are marked “xfail” (expected failure, not skipped) since they document a real, expected negative result, the real recovery sites falling outside the 3-sigma Monte Carlo region for both falls, rather than an untested gap.

Coverage spans the atmosphere model (checked against the COESA 1976 equations it is built from), the gravity model (varying with height), the dynamics right-hand side (correct descent sign, ablation switching off below the dark flight threshold), the hand-written fixed-step integrators (checked against a simple decay ODE), the adaptive integrator wrapper (event handling, dense output), the Monte Carlo HPD-threshold logic (checked against hand-computed examples and a known 2D Gaussian), the wind model, and an import check across every module.

5. Comparison and Optimization Work

The comparison set is fixed-step RK2 and RK4 at step sizes from 2 s down to 0.02 s against adaptive RK45 at normal tolerance, all benchmarked against the tight-tolerance DOP853 reference. Accuracy is impact-location error against the reference, cost is run time and number of function evaluations, and robustness is whether a method/step size combination completes the run at all.

Both fixed-step methods converge as the step size decreases. RK4 achieves comparable accuracy at a larger step size than RK2, consistent with its higher order.

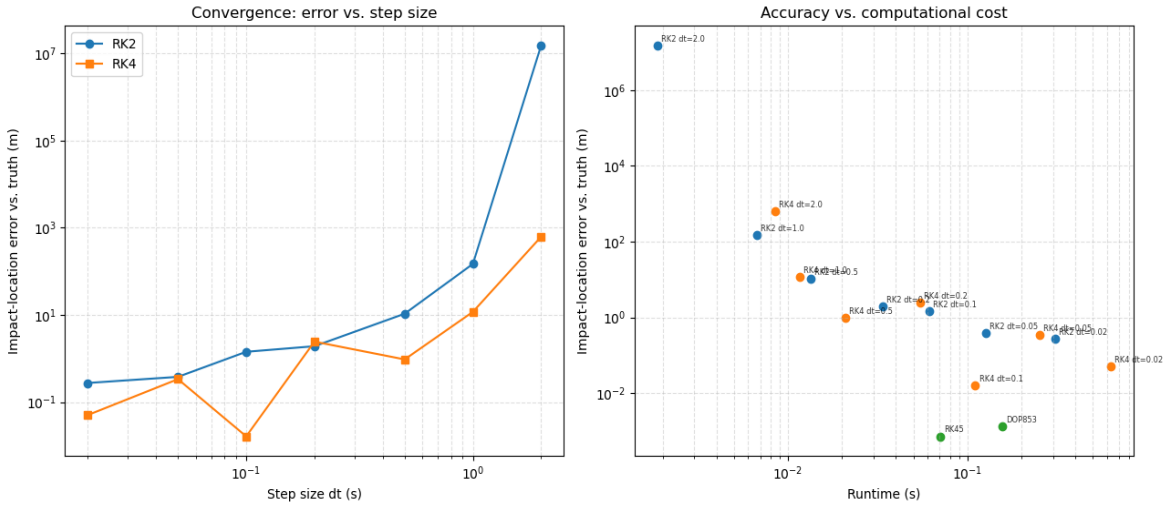


Figure 5: RK2/RK4 convergence, error vs. step size

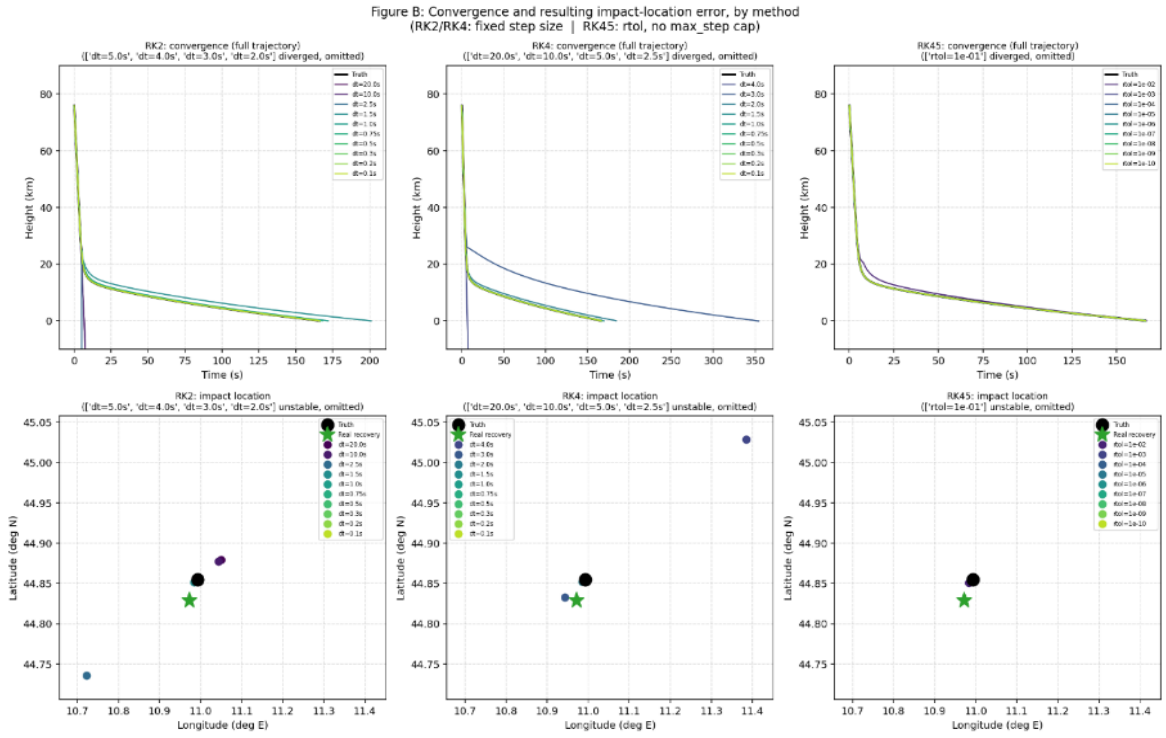


Figure 6: Convergence and resulting impact error for different methods RK2, RK4 and RK45.

Figures 5 and 6 compare convergence and impact-location error. RK45 stays close to the DOP853 reference at the tested tolerances, while RK2 and RK4 require sufficiently small steps to achieve similar accuracy.

Since the stiffness is localized to luminous flight, a piecewise stepping scheme was tested. A fine step was used during luminous flight and a coarse step during dark flight, compared against one uniform step size across the whole trajectory. This directly optimizes cost for a given accuracy. The piecewise scheme matches the accuracy of a fine uniform step while spending most function evaluations only where they are needed, cutting runtime for the same error tolerance.

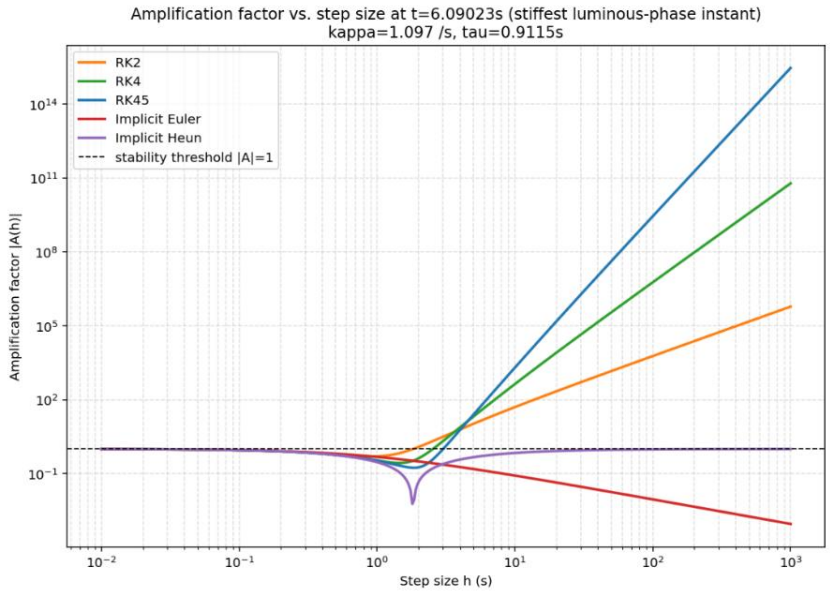


Figure 7: Stability behavior of explicit vs. implicit methods at the stiffest instant of luminous flight.

Figure 7 compares stability at the stiffest point of luminous flight, about 6.1 s after entry. The local decay rate is approximately 1.1 s^{-1}, giving a characteristic timescale of 0.91 s. The explicit methods exceed the stability threshold as the step grows and their amplification factors increase rapidly, while implicit Euler and implicit Heun remain stable over the tested range. This is the quantitative basis for piecewise stepping, implicit methods can safely take a much larger step once past this stiff point, so the step size can coarsen there instead of staying fine for the whole descent. Stability alone does not guarantee accuracy, so the impact-location error must also be checked.

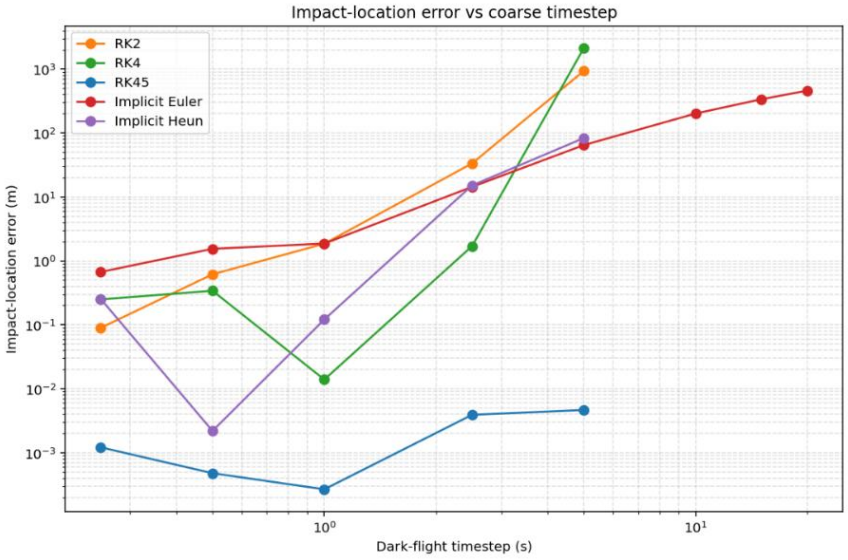


Figure 8: Meteor impact-location error versus dark-flight timestep for five numerical integration methods.

Figure 8 compares the effect of dark-flight step size on the final impact position. The explicit schemes become unstable at larger steps, while the implicit schemes remain stable over a wider range. However, a stable solution can still have substantial impact-location error, so the choice of step size should be based on both stability and convergence.

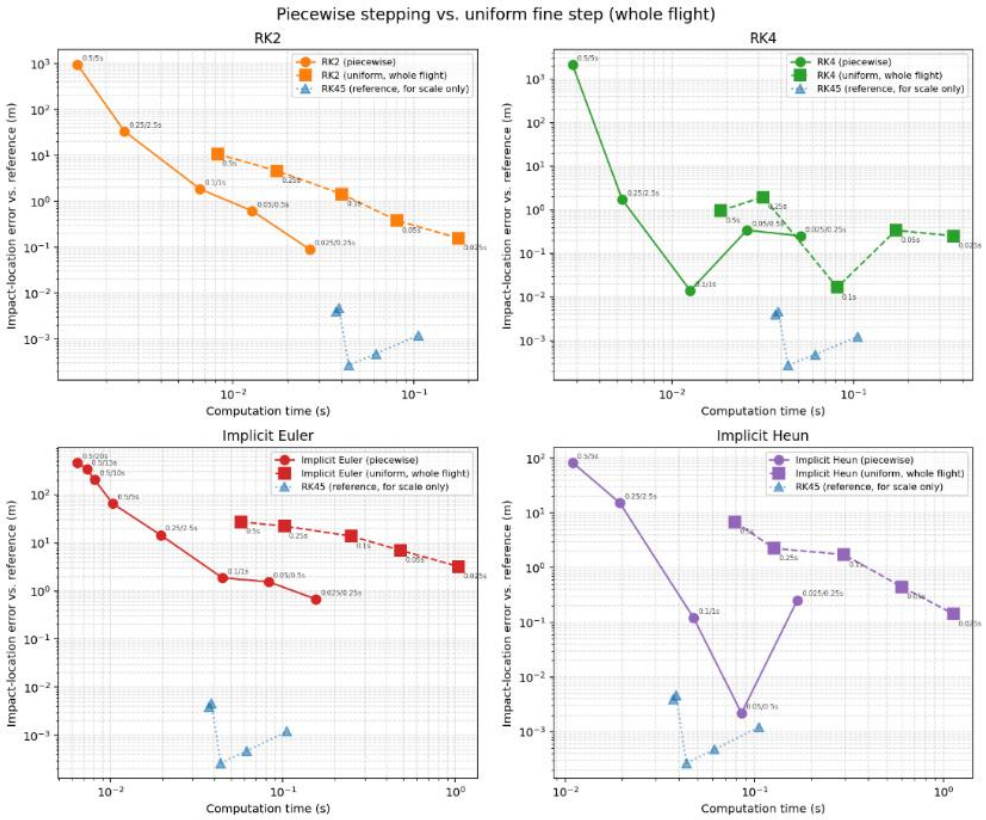


Figure 9: Piecewise-stepping vs. uniform-step accuracy/cost trade-off (RK45, the method carried forward, reaches the lowest error at a comparable runtime)

Figure 9 shows that piecewise stepping reduces runtime at similar impact-location error compared with a uniformly fine step.

RK2/RK4 were kept in the comparison rather than relying only on scipy's adaptive solvers, since they give a direct, interpretable baseline of how step size alone controls accuracy, which the adaptive methods hide behind their own step-size control. DOP853 is used only as the truth reference, standing in for the unavailable analytical solution. It is not treated as a candidate for this analysis. RK45 is the adaptive method actually carried forward for the trajectory and Monte Carlo runs reported in Sections 4 and 6. It was chosen over the fixed-step and piecewise alternatives because Figure 9 shows it reaching about 10^{-3} m error at a comparable runtime.

Beyond comparing integration methods, the per-step cost of the right-hand-side (RHS) function was profiled directly using cProfile for a function-level view and line_profiler for a line-by-line breakdown, at the same RK45 tolerance as the project's actual production runs, so these numbers match the method used throughout the project. Profiling showed the atmospheric density lookup (tabulated_density, using SciPy's interp1d) took about 58.8% of the RHS function's cumulative time, a disproportionate cost for a simple linear interpolation.

Interp1d is a general-purpose interpolator supporting multiple interpolation kinds, overhead wasted when called with a single scalar altitude on every RHS evaluation. Replacing it with NumPy's np.interp, a minimal linear-interpolation routine with far less per-call overhead, was tested as the fix (Table 3), and numerically verified to cut computational cost while keeping the output accuracy identical.

Table 3: interp1d vs. np.interp profiling comparison (runtime, cumulative time, density-lookup time, before/after/speedup).

Metric	Interp1d (Before)	Np.interp (After)	Change
Full run, 20x RK45 (wall-clock)	1.066 s	0.503 s	~2.1x faster
Meteor_rhs total time (cProfile cumtime, 20x RK45 runs)	0.811 s	0.371 s	~2.2x faster
Density lookup time (cProfile cumtime, 20x RK45 runs)	0.476 s	0.071 s	~6.7x faster
Density lookup's share of meteor_rhs time	58.8%	19.1%	~3.1x lower
Cavezzo predicted impact point	44.854870 N, 10.992439 E	44.854870 N, 10.992439 E	0

A Scalene profile measured peak memory use of 52.9 MB, mostly from library imports rather than trajectory storage. Function evaluations dominate solve_ivp runtime, so reducing the cost of the atmospheric density lookup was more useful than memory optimization.

As a rough estimate, each RHS call takes about 50 floating-point operations, or about 320 if sin, cos and the cube root are weighted as more expensive (about 30 to 40 times an add, from counted-float, n.d.). A production RK45 run makes about 2200 to 2400 RHS evaluations, so roughly 10^5 to 10^6 operations in total. At about 10 μs per call, this is only around 5×10^6 to 3×10^7 FLOP/s, far below hardware capability, so the run is limited by Python overhead and not arithmetic. This matches the profile above, where a simple interpolation took 58.8% of the RHS time. The arithmetic intensity is low, about 0.5 FLOP per byte of state data, but this works on a few hundred bytes that stay in cache, so memory traffic is not the limit.

6. Limitations and Lessons Learned

The simplifications listed in Section 1 are the most likely explanation for why the predicted impact region misses the real recovery site for both falls, since real meteoroids fragment and tumble in ways this model cannot capture, so some of the gap comes from physics the model does not represent, not from numerical error.

On top of that, the drag and ablation coefficients are not fitted per fall, exact values for Bunburra or Cavezzo were not available, so generalized values from Ceplecha et al. (1998) were used for both. A one at a time sensitivity sweep on Cd and sigma_abl (full reported range for sigma_abl, plus or minus 10 percent for Cd, since no published range exists) shifts the Cavezzo impact point by at most 0.19 km and Bunburra's by at most 0.72 km, under 6 percent of the 3.31 km Cavezzo miss and under 4 percent of the 21.07 km Bunburra miss, so this generalization is real but does not explain most of the mismatch for either fall.

Dark-flight wind was tested for Cavezzo using the bearing and speed reported in Gardiol et al. (2021, Sec 3.4), but it moved the predicted impact point 1.2 km in the wrong direction, increasing the miss from 3.31 km to 3.82 km rather than closing it. Working backward, closing the gap exactly would need an average wind of 21.23 m/s over the 156 s of dark flight, physically plausible since the paper reports 20 to 28 m/s at the relevant altitudes, but the required bearing (209 degrees) is close to the opposite of the reported bearing (103 degrees) actually used. So, the remaining miss is not explained by wind alone and is more likely several simplifications stacking together rather than one dominant effect.

Only two meteorite falls were tested, both using the same default coefficients. This limits any claim that the model would generalize to other falls. The recovery mismatch also cannot be assigned to one physical

assumption without modelling fragmentation, winds and other effects separately. The parameter sensitivity tests narrow the likely causes but do not prove which omitted effect dominates.

The biggest lesson was telling apart three different sources of error that could easily get lumped into one mismatch against the real recovery site. Numerical error was checked against the DOP853 reference and found small, and parameter uncertainty was checked with the sensitivity sweep above and also found small. What is left over is model form error, physics the equations simply do not include, like fragmentation, lift and a more detailed wind profile, the most likely explanation once the first two are ruled out with real numbers, even though it was not directly modelled here. Section 5's stability analysis also showed the stiffness sits mostly in luminous flight, why piecewise stepping was tried instead of one step size for everything. The comparison also showed that piecewise stepping is not the only way to handle rapid changes during luminous flight. A sufficiently fine adaptive method can reduce its step size automatically where the trajectory changes quickly, while an implicit method offers better stability at larger steps. For this project RK45 was retained because it gave low impact-location error at similar computational cost in the measured comparison. The main lesson is to select a method using both numerical accuracy and runtime rather than stability alone. In practice, a 3.3 km miss for Cavezzo would still point a search team to the right area, whereas a 21 km miss for Bunburra would not have helped.

7. AI-Use Reflection

I used AI (Claude, Sonnet 5) to generate most of the project's code from my own specifications, debug environment issues, set up the GitHub workflow, and draft this report from the project's code and tests, since these tasks were time-consuming and mostly mechanical, not because they needed engineering judgment. The engineering decisions stayed mine, the physical model, which integration methods to implement, and the Monte Carlo design (including the 1/2/3-sigma HPD convention), though I chose to also implement and compare the piecewise and implicit methods to see the trade-off directly (Section 5). I reviewed and tested every piece of code and every commit locally before accepting it, and the AI assistant never committed or pushed on its own. Using it for the repetitive debugging and documentation work freed up my time to focus on those engineering decisions, the validation design, and interpreting results. By volume, AI generated about 90 percent of the code but the engineering decisions, the physical model, and the interpretation behind every validation result were mine.

I checked AI output against actual behavior, not accepted on explanation alone, by testing locally and pasting real console output back after each change. The most significant catch was generated code using physical values without flagging them as unbacked assumptions, fixed by substituting the real values from the source papers (Spurny et al. 2012, Gardiol et al. 2021, Ceplecha et al. 1998), reinforcing the same standard applied throughout. A second catch was a convergence-order test that measured single-step error instead of error accumulated over many fixed steps to a set end time, which silently fits an order one too high. It looked right until I checked the fitted exponent against the known theoretical order for RK2 and RK4 and it was off, so I worked with Claude to fix it. Having to check every generated number against the source papers before trusting it is what actually built my understanding of the physics, a plausible-looking result was not enough, I had to know what each term in the equations represented to tell whether a number made sense.

I noticed that a lot of Monte Carlo sample points were visibly outside the 3-sigma contour, when only about 1.1% should be. The existing tests passed because they only checked the code against itself, so I had the function checked against a 2D Gaussian with a known answer. That exposed the bug: hpd_levels was summing density values instead of counting samples, so the nominal 39.3/86.5/98.9% contours only enclosed about 23/65/90% of samples. I worked through the fix with AI, verified it against the same Gaussian, and added that check as a permanent test.

8. References

1) Spurny, P., Bland, P. A., Shrbeny, L., Borovicka, J., Ceplecha, Z., et al. (2012). The Bunburra Rockhole meteorite fall in SW Australia: fireball trajectory, luminosity, dynamics, orbit, and impact position from photographic and photoelectric records. Meteoritics & Planetary Science, 47(2), 163-185. https://doi.org/10.1111/j.1945-5100.2011.01321.x
2) Ceplecha, Z., Borovicka, J., Elford, W. G., ReVelle, D. O., Hawkes, R. L., Porubcan, V., & Simek, M. (1998). Meteor phenomena and bodies. Space Science Reviews, 84(3-4), 327-471.
3) Gardiol, D., Barghini, D., Buzzoni, A., et al. (2021). Cavezzo, the first Italian meteorite recovered by the PRISMA fireball network. Orbit, trajectory, and strewn-field. Monthly Notices of the Royal Astronomical Society, 501(1), 1215-1227. https://doi.org/10.1093/mnras/staa3646
4) U.S. Committee on Extension to the Standard Atmosphere (COESA). (1976). U.S. Standard Atmosphere, 1976. U.S. Government Printing Office.
5) counted-float. (n.d.). FLOP weights. Retrieved October 22, 2026, from https://counted-float.readthedocs.io/en/stable/flop_weights/